



VIT BHOPAL UNIVERSITY

School of Computing Science Engineering and Artificial Intelligence

PROJECT REPORT

Library Management Portal
(Console Based Library System)

Course: CSE2006 - Programming in Java
VITyarthi - Build Your Own Project

Submitted By	Guided By
Name: Krish Pandey	Dr. Sandeep Monga
Programme: B.Tech CSE (Artificial Intelligence and Machine Learning)	Faculty, School of Computing Science Engineering & AI
Registration No: 25BAI11066	

Academic Year 2025 - 2026
VIT Bhopal University, Madhya Pradesh, India

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions
9. Technology Stack
10. Implementation Details
11. Screenshots and Sample Output
12. Testing
13. Challenges Faced
14. What I Learned
15. Future Improvements
16. Conclusion
17. References

1. INTRODUCTION

A library has many books and many members. Managing all of this by hand is hard. We can lose track of which book is issued, who has which book, and how much fine a person has to pay. So I made a simple Library Management Portal in Java. It is a console based program where a librarian and a member can do their work through menus.

The librarian can add or remove books and members. The member can see available books, issue a book, return a book, and pay fine. The system also calculates fine automatically if a book is returned late. I used Object Oriented Programming concepts, ArrayList, PriorityQueue, and ScheduledExecutorService in this project.

This project is for the course CSE2006 Programming in Java under the VITyarthi Build Your Own Project at VIT Bhopal University.

2. PROBLEM STATEMENT

In a normal library, everything is done by hand. This creates many problems:

- It is hard to know which books are free and which are already issued.
- Fine is calculated by hand, so mistakes can happen.
- There is no limit on how many books one person can take.
- Same person can register twice with same phone number.
- Librarian does not get a clear list of all members and their dues.
- Records can get lost or become messy when the library grows.

I wanted to make a small software that solves these problems. The program should have clear menus, check wrong inputs, and follow simple rules like maximum 2 books per member.

3. OBJECTIVES

Main goals of this project:

- Make a working Library Management System using pure Java.
- Give different options to Librarian and Member.
- Add, remove, and list books and members properly.
- Allow only 2 books per member and charge fine after 10 days (Rs 3 per day).
- Check that name has only letters and phone number has exactly 10 digits.
- Use ArrayList, PriorityQueue, and timer for fine calculation.

Extra goals:

- Keep the menus simple and show clear error messages.
- Make sure issuedCopies and fine stay correct after every action.
- Write proper documentation for the project.

4. FUNCTIONAL REQUIREMENTS

The project has three main parts:

4.1 Librarian Module

- **Register Member:** Take name, age and phone. Give a unique Member ID. Do not allow same phone number twice.
- **Remove Member:** Remove by ID. Also free any books that person had taken.
- **Add Book:** Take title, author and number of copies. Each copy gets its own Book ID.
- **Remove Book:** Remove by Book ID. Also remove it from any member who had issued it.

- **View All Members:** Show ID, name, age, phone, books taken and fine amount.
- **View All Books:** Show Book ID, title, author and how many copies are still free.

4.2 Member Module

- **List Available Books:** Show only those books that still have free copies.
- **List My Books:** Show the books I have currently issued.
- **Issue Book:** Can issue only if fine is zero and I have less than 2 books. Select book by ID.
- **Return Book:** Return by Book ID. If late, fine is added. Free the copy again.
- **Pay Fine:** Show total fine and clear the balance after payment.

4.3 Main Menu and Login

- First menu: Enter as Librarian, Enter as Member, or Exit.
- Member login needs Name and Phone number. It checks if that person is registered.
- Every menu has a Back option so we can go back safely.
- Wrong inputs show a clear error message and the menu comes again.

5. NON-FUNCTIONAL REQUIREMENTS

Point	What it means in this project
Usability	Simple numbered menus, clear messages, and prompts for every input.
Reliability	Wrong input does not crash the program. Rules like max 2 books are always checked.
Maintainability	Code is divided into separate classes: Book, Member, IssuedBook, Library, Librarian, Main.
Performance	ArrayList and PriorityQueue are fast enough for this size. Fine timer runs in background.
Error Handling	Invalid name, phone, age or menu choice is caught and a message is shown.
Resource Use	Deleted IDs are reused so the ID numbers do not grow forever.

6. SYSTEM ARCHITECTURE

The program is divided into three simple layers:

1. Presentation Layer (Main class)

This is the part the user sees. It shows menus, takes input, checks if the input is correct (isAlpha and isPhone methods), and then calls the right function in Librarian or Member.

2. Business Logic Layer

Librarian class does all admin work like register, remove, add book, list etc. Member class does issue, return, pay fine and list personal books. IssuedBook class keeps the timer for fine.

3. Data Layer

Library class just holds two lists - one for all books and one for all members. Book class stores title, author, total copies, issued copies and ID.

Layer	Classes	Work
Presentation	Main	Menus, input check, flow control
Business Logic	Librarian, Member, IssuedBook	Rules, issue/return, fine timer
Data	Library, Book	Store books and members

Table 1: Layers of the system

7. DESIGN DIAGRAMS

7.1 Use Case (What each person can do)

Librarian can: Register Member, Remove Member, Add Book, Remove Book, View All Members, View All Books.

Member can: Login, List Available Books, List My Books, Issue Book, Return Book, Pay Fine.

System does: Calculate fine every second after 10 days free period using ScheduledExecutorService.

7.2 Class Relationships

Main uses Librarian, Library and Member.

Librarian manages Library (adds and removes Books and Members).

Librarian also uses PriorityQueue of Integer for ID reuse.

Library contains ArrayList of Book and ArrayList of Member.

Member has ArrayList of IssuedBook (composition).

Member also refers to Library when issuing a book.

IssuedBook has a Book (association) and runs ScheduledExecutorService for fine timer.

Book has: title, author, totalCopies, issuedCopies, ID.

Member has: name, age, phoneNo, booksBorrowed, balance, ID.

7.3 How the program flows

1. Program starts. Library and Librarian objects are created. Main menu is shown.
2. User chooses 1 for Librarian, 2 for Member, or 3 to Exit.
3. If Librarian: sub-menu appears. User can register/remove member, add/remove book, or list things. After each action the sub-menu comes back.
4. If Member: user enters Name and Phone. Program checks if that member exists. If yes, member menu opens. If no, error is shown.
5. Issue Book: First check fine is 0 and less than 2 books already taken. Then list books, take Book ID, create IssuedBook (starts timer), increase issuedCopies.

6. Return Book: Show borrowed books, take ID, calculate fine from timer, add fine to balance, remove from list, decrease issuedCopies.
7. Pay Fine: Show total fine and clear the balance.

7.4 Sequence for Issue Book

Member chooses Issue Book in Main.
Main asks Member for fine amount.
Main checks how many books Member already has.
Main asks Librarian to list all books.
Main calls Member.issueBook with the Book ID.
Member gets all books from Library and finds the matching ID.
Member checks if free copies are left.
Member creates a new IssuedBook (this starts the fine timer).
Member increases issuedCopies of that book.
Success message is shown.

8. DESIGN DECISIONS

Here are some important choices I made while coding and why I made them:

Each copy is a separate Book object

When librarian adds 5 copies, the program creates 5 Book objects, each with its own ID. This makes it easy to track which exact copy is issued.

PriorityQueue for ID reuse

When a member or book is removed, its ID goes into a PriorityQueue. Next time we register someone, we first use the smallest free ID. This way IDs do not keep growing forever.

ScheduledExecutorService for fine

Every IssuedBook starts a small timer that runs every second. After 10 seconds the fine starts increasing by Rs 3. This shows concurrent programming. In real life we would use days, but seconds are good for testing.

Maximum 2 books only

This is a simple business rule. Before issuing, the program checks if the member already has 2 books.

Fine must be zero before new issue

If a member has unpaid fine, he cannot take a new book. This pushes people to return books on time and pay fine.

Single Java file

All classes are in one file so it is easy to compile and submit. Main is public and other classes are package-private.

9. TECHNOLOGY STACK

Category	What I used	Why
Language	Java SE	Main language of the course
Collections	ArrayList, PriorityQueue	Store books/members and reuse IDs
Concurrency	ScheduledExecutorService	Fine timer runs in background
Input	BufferedReader, Scanner	Read from keyboard
Validation	Regex and Character.isDigit	Check name and phone
Build	javac and java	Compile and run from terminal

Table 2: Technology used in the project

This project has no external libraries. It is pure Java so the focus stays on OOP and core language features.

10. IMPLEMENTATION DETAILS

10.1 Classes in the project

Main

This is the starting point. It has `isAlpha()` and `isPhone()` to check inputs. It has `enterLibrarian()` and `enterMember()` for the two menus. The `main()` method creates Library and Librarian and shows the first menu.

Book

Stores title, author, totalCopies, issuedCopies and ID. Has simple getter methods.

IssuedBook

Wraps a Book and keeps timeElapsed and fine. When created, it starts a ScheduledExecutorService that increases time every second. After 10 seconds, fine becomes $(\text{timeElapsed} - 10) * 3$.

Member

Stores name, age, phone, list of IssuedBook, balance and ID. Has methods for issueBook, returnBook, listBorrowedBooks, listAvailableBooks, calFine and payFine.

Library

Just two ArrayLists - one for books and one for members. Has getters to return those lists.

Librarian

Does all admin work. Uses two PriorityQueues and two static counters for Member ID and Book ID. Has methods to add/remove book, register/remove member, and list everything.

10.2 Important logic**Fine calculation:**

When an IssuedBook is created, a timer starts. Every second timeElapsed goes up by 1. When it becomes more than 10, fine is set to $(\text{timeElapsed} - 10) * 3$. This runs in background and does not stop the menu.

ID reuse:

When we remove a member or book, the free ID is put into a PriorityQueue. Next time we add something, if the queue is not empty we take the smallest free ID. Otherwise we use the next number.

Checks before issuing:

Before a member can issue a book, three things are checked: fine must be 0, number of already borrowed books must be less than 2, and the selected book must have free copies left.

10.3 How to run

Compile: `javac LibraryPortal.java`

Run: `java Main`

11. SCREENSHOTS AND SAMPLE OUTPUT

Below are real screenshots taken while running the program. They show the main flow of the Library Portal.

11.1 Program starts - Main Menu

```
PS C:\Users\Krish> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Krish\AppData\Local\Temp\vscodews_8c1
88\jdt_ws\jdt.ls-java-project\bin' 'Main'
Library Portal Initialized...
1. Enter as Librarian
2. Enter as a Member
3. Exit
```

Figure 1: Library Portal Initialized and main menu appears

11.2 Enter as Librarian - Librarian Menu

```
1
-----
1. Register a Member
2. Remove a Member
3. Add a Book
4. Remove a Book
5. View all Members along with their Books and Fines to be Paid
6. View all Books
7. Back
-----
```

Figure 2: Librarian menu with all options

11.3 Register a Member

```
1
-----
Name : krish
Age : 20
Phone No : 9263402355
-----
Member Successfully Registered with Member ID : 1!!
-----
1. Register a Member
2. Remove a Member
3. Add a Book
4. Remove a Book
5. View all Members along with their Books and Fines to be Paid
6. View all Books
7. Back
-----
```

Figure 3: Member 'krish' registered successfully with Member ID 1

11.4 Add a Book

```
3
-----
Title of the Book : happy poter
Author of the Book : jk
Total Copies : 5
-----
The Book - 'happy poter' added Successfully!!
-----
1. Register a Member
2. Remove a Member
3. Add a Book
4. Remove a Book
5. View all Members along with their Books and Fines to be Paid
6. View all Books
7. Back
-----
```

Figure 4: Book 'happy poter' added with 5 copies

11.5 Enter as Member - Login

```

2
-----
Name : krish
Phone No : 9263402355

Welcome krish!!
Member ID : 1

-----
1. List Available Books
2. List My Books
3. Issue Book
4. Return Book
5. Pay Fine
6. Back
-----

```

Figure 5: Member login successful - Welcome message and Member menu

11.6 List Available Books (before adding)

```

-----
3
-----
No Books in the Library Currently!
-----
1. List Available Books
2. List My Books
3. Issue Book
4. Return Book
5. Pay Fine
6. Back
-----

```

Figure 6: Message when no books are present in the library

These screenshots cover the main working of the program: starting the portal, registering a member, adding a book, logging in as member, and checking available books. The same menus are used for issue, return and pay fine operations as explained in the earlier sections.

12. TESTING

I tested the program by running it many times and trying both correct and wrong inputs.

What I tested	What I did	What happened	Result
Register member	Name=krish, Age=20, Phone=9263402355	Member created with ID 1	Pass
Wrong name	Name with numbers	ERROR: Invalid Name	Pass
Wrong phone	Phone less than 10 digits	ERROR: Invalid Phone Number	Pass
Duplicate phone	Same phone again	Duplicate Registration message	Pass
Issue with fine	Try issue after late return	Blocked - due amount message	Pass

Issue 3rd book	Already have 2 books	Blocked - max 2 books	Pass
Return wrong book	Book ID not in my list	You have not Issued any Book	Pass
ID reuse	Remove member 1, register new	New member gets ID 1 again	Pass

Table 3: Test cases I tried

13. CHALLENGES FACED

- Making the fine timer work in the background without stopping the menu was a bit hard. I learned how ScheduledExecutorService works.
- Keeping issuedCopies correct after every issue, return, remove member and remove book needed careful coding.
- Mixing Scanner and BufferedReader sometimes left extra newlines. I had to be careful with the order of reading.
- Putting all classes in one file while keeping the code readable took some time.

14. WHAT I LEARNED

- How to divide a real problem into different classes (Book, Member, Library, Librarian).
- How to use ArrayList to store a list of objects and PriorityQueue to reuse IDs.
- How to run a timer in the background using ScheduledExecutorService.
- Why input validation is important so the program does not crash on wrong data.
- How to write simple rules in code (max 2 books, fine must be zero).
- How to make a menu driven console program that feels easy to use.

15. FUTURE IMPROVEMENTS

- Save data in a file or database so it does not disappear when we close the program.
- Change the fine timer from seconds to real days.
- Make a graphical interface using JavaFX or Swing.
- Add search by book title or author.
- Add login for librarian also with password.
- Write proper unit tests using JUnit.

16. CONCLUSION

I successfully made a working Library Management Portal in Java. It has separate options for librarian and member, checks wrong inputs, follows rules like maximum 2 books, and calculates fine automatically. The code is divided into clear classes and uses ArrayList, PriorityQueue and ScheduledExecutorService. This project helped me practice Object Oriented Programming and console based application development for the CSE2006 course.

The project meets the VITyarthi requirements: three main modules, clear input output, logical flow, non-functional points, and proper documentation. I can run it, test it, and explain every part of the code.

17. REFERENCES

- [1] Oracle Java Documentation - <https://docs.oracle.com/javase/>
- [2] Java ScheduledExecutorService API documentation
- [3] Herbert Schildt - Java: The Complete Reference
- [4] CSE2006 Programming in Java course material, VIT Bhopal University

[5] VITyarthi Build Your Own Project guidelines